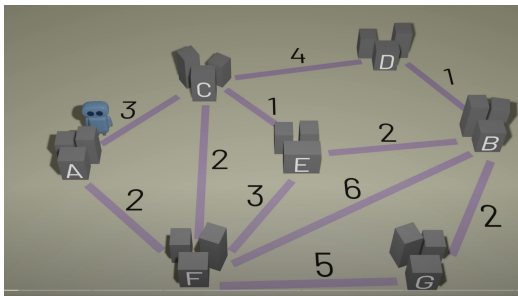


Shortest Path Algorithms

Graph Theory

August 11, 2025



Outline

- Video
- Problem Statement
- Dijkstra's Algorithm
- Examples
- Efficiency
- Software Solutions

https://www.youtube.com/watch?v=EFg3u_E6eHU&ab_channel=SpanningTree

How Dijkstra's Algorithms Works by Spanning Tree

Problem Statement

- When using applications like Google Maps, you're looking for a **shortest path**
- The street maps are represented as graphs
- Edge weights represent estimated driving times
- While simple cases can be solved by guess-and-check, we need systematic approaches for complex problems
- We need **algorithms** - step-by-step procedures for solving problems

Shortest Path Problem: Definition

The Shortest Path Problem

Given a network $G = (V, A)$ with costs c_{ij} on the arcs, find the path from a source node s to a target node t that minimizes the total cost.

Sets

- V : Set of nodes in the network
- A : Set of arcs in the network

Parameters

- c_{ij} : Cost (or distance) of traversing arc $(i, j) \in A$
- s : Source node
- t : Target node

Variables

- x_{ij} : flow along arc (i, j) on the shortest path

Shortest Path Problem: Mathematical Model

Objective Function

Minimize the total cost of the selected path:

$$\text{Minimize} \quad \sum_{(i,j) \in A} c_{ij} x_{ij}$$

Constraints

Flow Conservation:

$$\sum_{j:(i,j) \in A} x_{ij} - \sum_{j:(j,i) \in A} x_{ji} = \begin{cases} 1 & \text{if } i = s \\ -1 & \text{if } i = t \\ 0 & \text{otherwise} \end{cases} \quad \forall i \in V$$

Non-negativity Constraints:

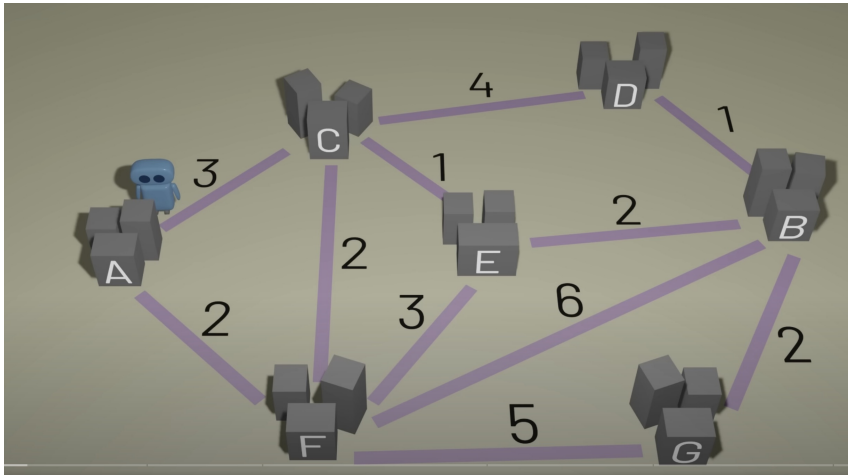
$$x_{ij} \geq 0 \quad \forall (i,j) \in A$$

Applications

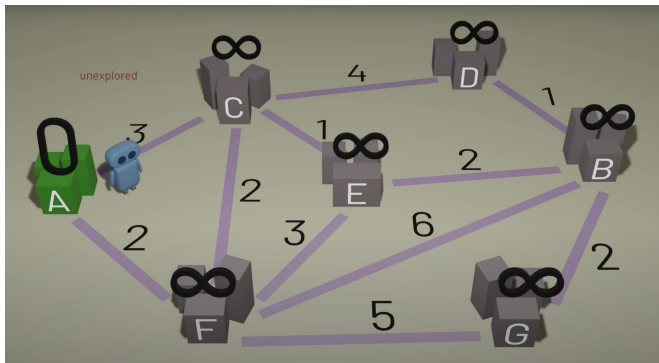
- Route planning
- Network routing
- Logistics optimization
- Social network analysis

Dijkstra's Algorithm

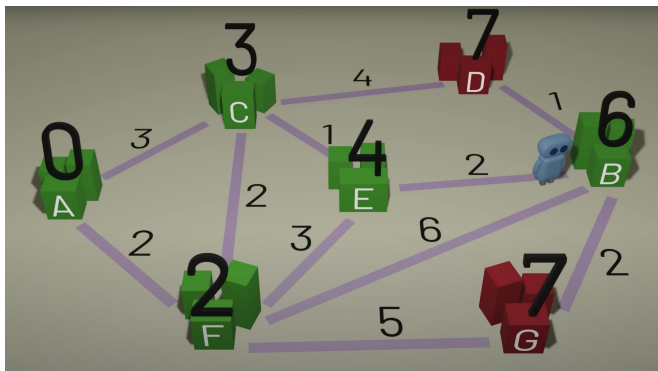
- 1 Mark the starting vertex with a distance of zero. Designate this vertex as current.
- 2 Find all vertices leading to the current vertex. Calculate their distances to the start.
- 3 Mark the current vertex as visited. We will never look at this vertex again.
- 4 Mark the vertex with the smallest distance as current, and repeat from step 2.



How Dijkstra's algorithm works (YouTube)

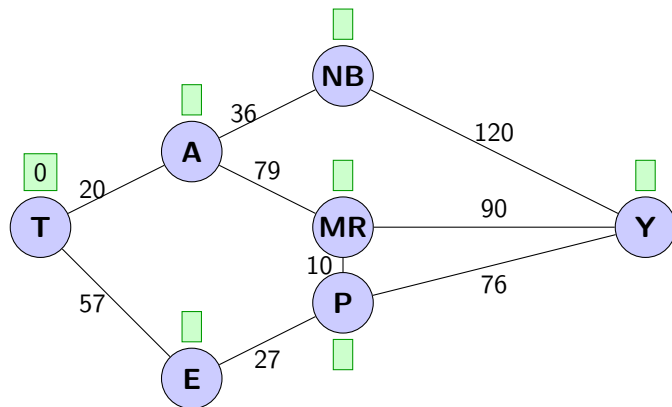


current	a	f	c	e	d	g	b

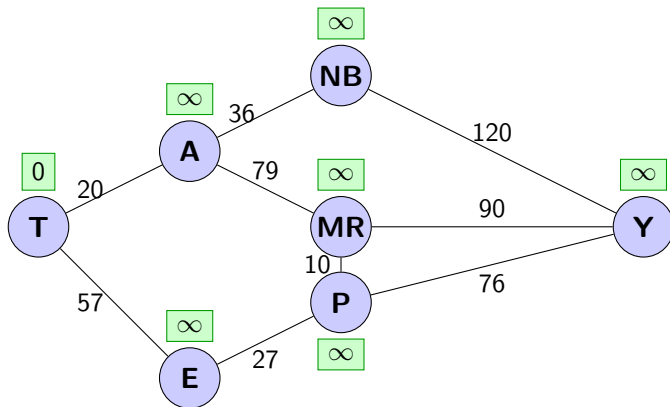


current	a	f	c	e	d	g	b
a	0	2	3	∞	∞	∞	∞
f	-	-	3	5	∞	7	∞
c	-	-	-	4	7	7	∞
e	-	-	-	-	7	7	6

Example 1: Yakima to Tacoma

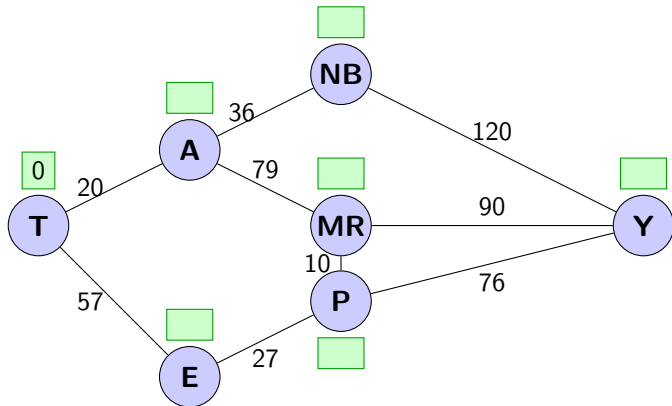


Example 1: Yakima to Tacoma

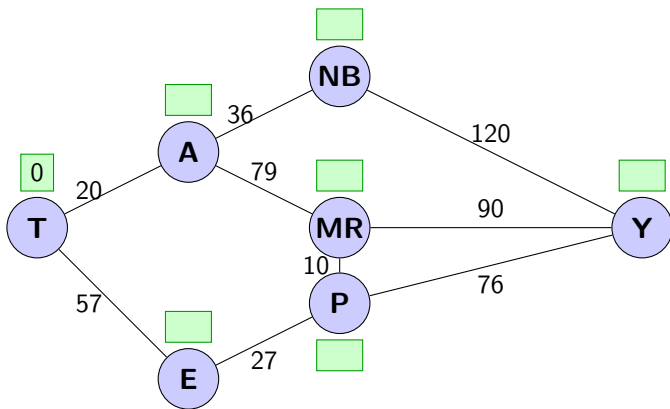


- Finding shortest path from Tacoma (T) to Yakima (Y)
- Graph shows travel times in minutes

- How Dijkstra's algorithm works (YouTube)
- Python Example using Networkx

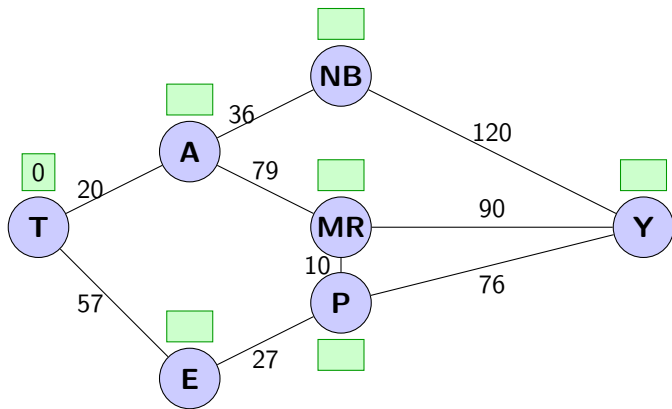


current	T	A	E	NB	MR	P	Y

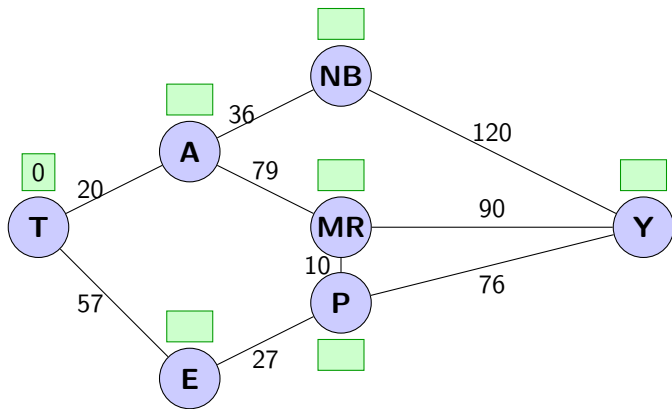


- 1 starting vertex = current, dist = 0.
- 2 Find all leading to the current vertex. Calculate distances to the start.
- 3 Mark the current vertex as visited. We will never look at this vertex again.
- 4 Mark vertex with smallest distance as current, and repeat 2.

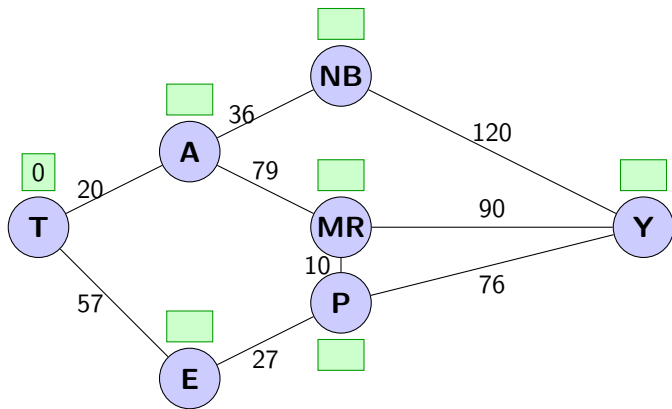
current	T	A	E	NB	MR	P	Y



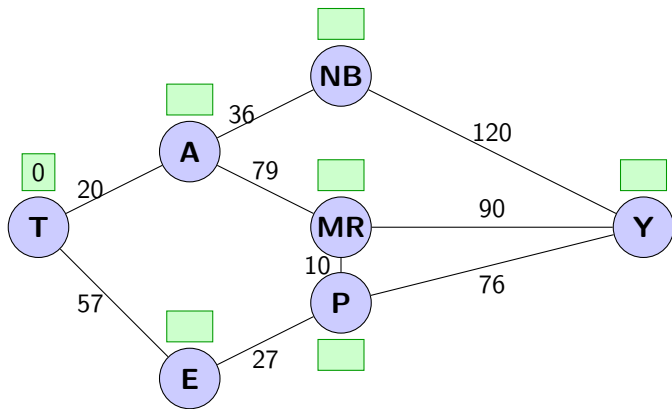
current	T	A	E	NB	MR	P	Y



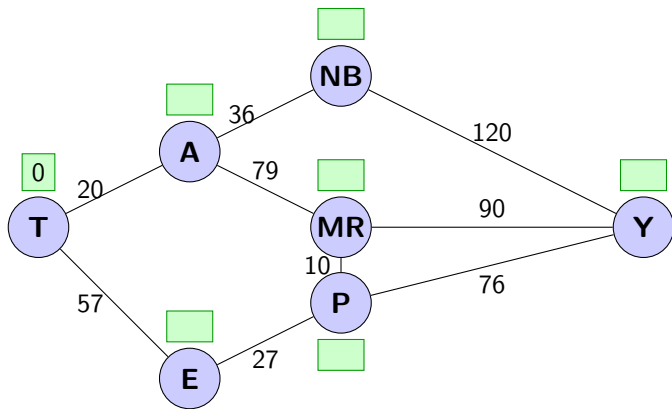
current	T	A	E	NB	MR	P	Y



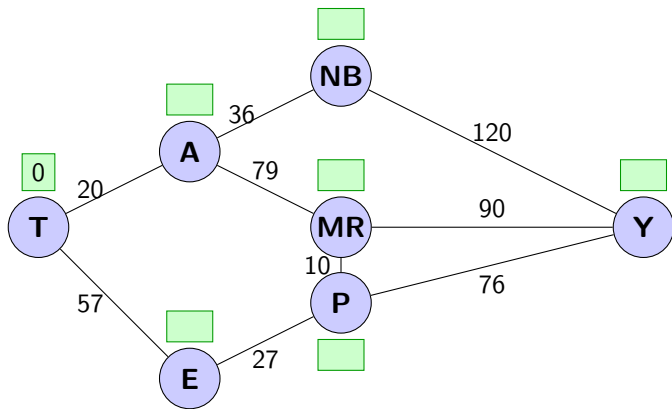
current	T	A	E	NB	MR	P	Y



current	T	A	E	NB	MR	P	Y



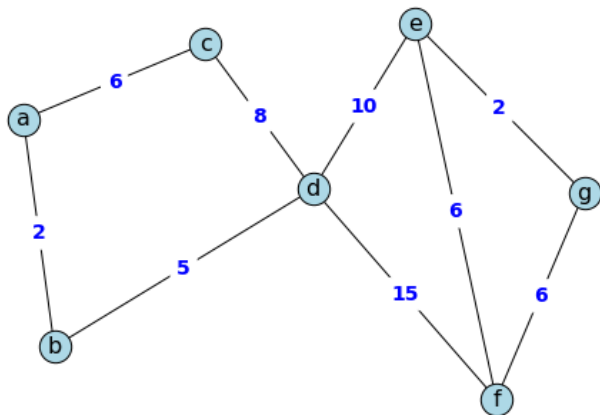
current	T	A	E	NB	MR	P	Y



current	T	A	E	NB	MR	P	Y

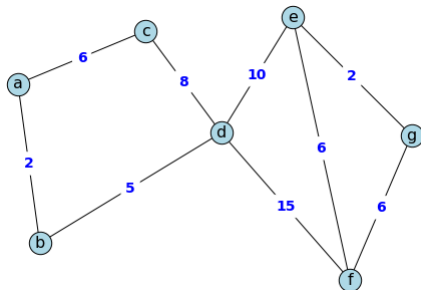
current	T	A	E	NB	MR	P	Y

Example 2: Node a to g



- We want to find the shortest path from node a to node g
- Initialize algorithm at node 'a'

Example 2: Table to work with

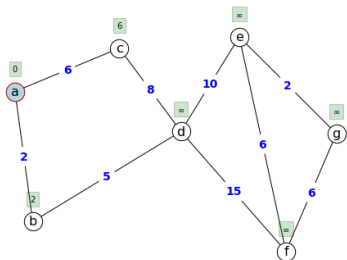


current	a	b	c	d	e	f	g

- 1 starting vertex = current, dist = 0.
- 2 Find all leading to the current vertex. Calculate distances to the start.
- 3 Mark the current vertex as visited. We will never look at this vertex again.
- 4 Mark vertex with smallest distance as current, and

repeat 2.

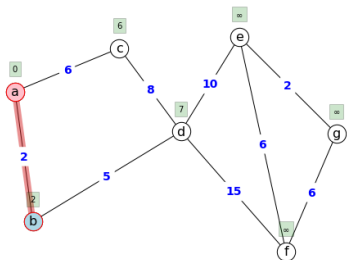
Example 2: First Steps



current	a	b	c	d	e	f	g
a	0	2	6	∞	∞	∞	∞

- Mark a as current, compute distances to adjacent nodes

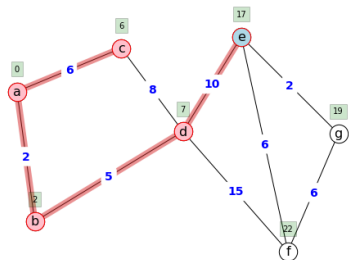
Example 2: Next Steps



current	a	b	c	d	e	f	g
b	0	2	6	7	∞	∞	∞

- Mark b as current (lowest distance), compute distances

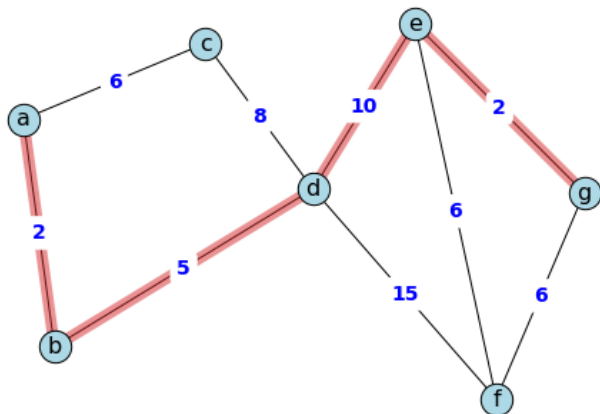
Example 2: Later Steps



current	a	b	c	d	e	f	g
e	0	2	6	7	17	22	19

- Continue marking vertices as visited and computing distances

Example 2: Solution



- The shortest path from a to g is a - b - d - e - g
- Path length = $2 + 5 + 10 + 2 = 19$

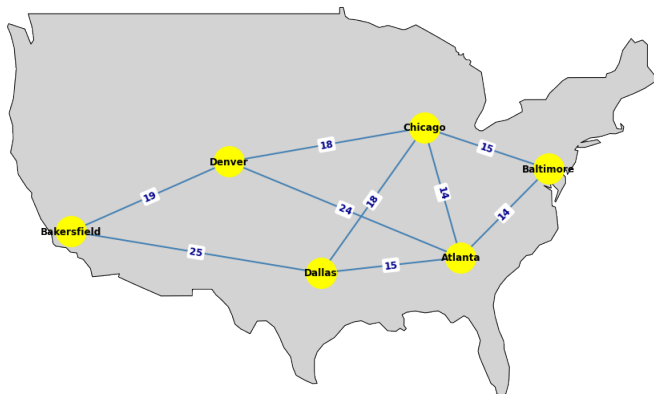
Efficiency of Dijkstra's Algorithm

- Dijkstra's algorithm is an **optimal algorithm**
 - It always produces the actual shortest path
- It is also **efficient**
 - Takes approximately V^2 calculations (V = number of vertices)
 - A graph with 100 vertices: 10,000 calculations
 - Fast enough for GPS navigation systems
- In contrast, listing all possible paths would be **inefficient**
 - 25 vertices could require 10^{25} calculations
 - World's fastest computer: over 1000 years!

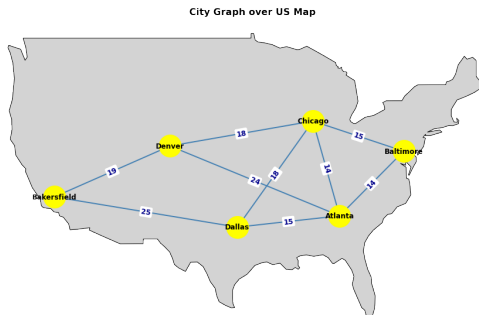
Example 3: Shipping Routes

- A shipping company needs to find the fastest route from Baltimore to Bakersfield
- Travel times between processing centers in hours (including 3-hour processing time)

City Graph over US Map

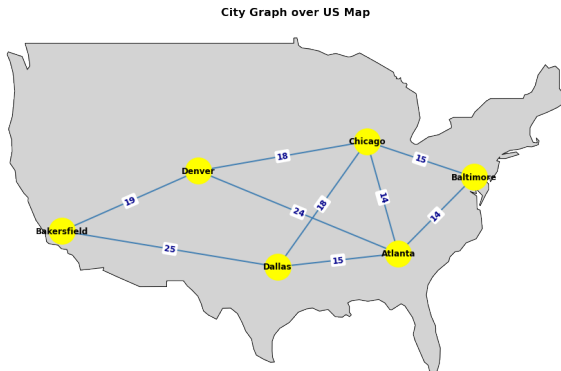


Example 3: Shipping Routes - Initialization



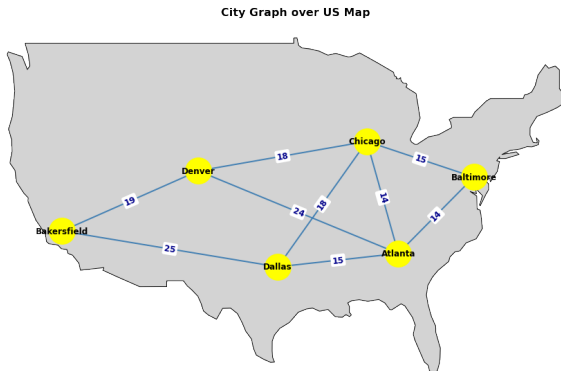
current	Baltimore	Chicago	Atlanta	Dallas	Denver	Bakersfield
Baltimore	0	15	14	∞	∞	∞

Example 3: Shipping Routes - Steps...



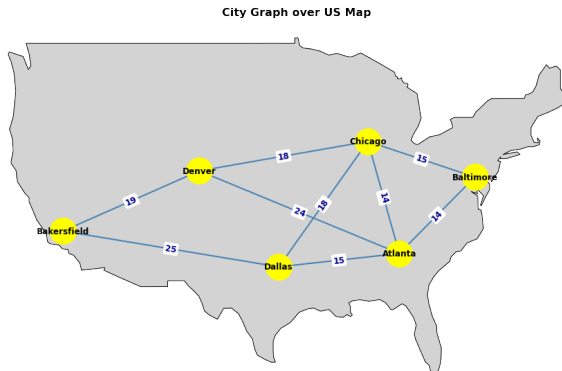
current	Baltimore	Chicago	Atlanta	Dallas	Denver	Bakersfield

Example 3: Shipping Routes - Steps...



current	Baltimore	Chicago	Atlanta	Dallas	Denver	Bakersfield

Example 3: Shipping Routes - Steps...



current	Baltimore	Chicago	Atlanta	Dallas	Denver	Bakersfield

Example 3: Shipping Routes

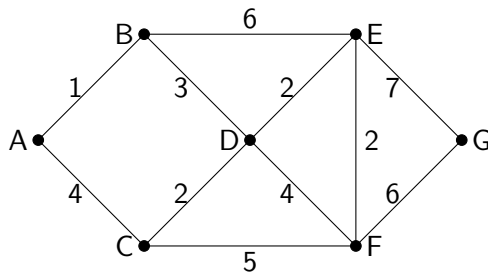
- A shipping company needs to find the fastest route from Baltimore to Bakersfield
- Travel times between processing centers in hours (including 3-hour processing time)
- We can work directly from a distance table

	Baltimore	Denver	Dallas	Chicago	Atlanta	Bakersfield
Baltimore	*			15	14	
Denver		*		18	24	19
Dallas			*	18	15	25
Chicago	15	18	18	*	14	
Atlanta	14	24	15	14	8	
Bakersfield		19	25			*

current	Baltimore	Chicago	Atlanta	Dallas	Denver	Bakersfield
Baltimore	0	15	14	∞	∞	∞

Practice Exercise

Find the shortest path between vertices A and G in this graph:



- NetworkX (Python library)
 - Easy graph creation and manipulation
 - Built-in shortest path algorithms
 - Visualization capabilities
- Other options:
 - igraph (Python, R, C++)
 - JUNG (Java)
 - Graph-tool (Python)
 - Specialized GIS software for real-world routing problems

Summary

- Shortest path algorithms find optimal routes between vertices
- Dijkstra's algorithm:
 - Starts at the end vertex (or beginning, in reverse)
 - Systematically explores the graph
 - Marks vertices as visited
 - Always selects the unvisited vertex with the smallest distance
- It is both optimal and efficient
- Applications: routing, navigation, network analysis